Fixed-Point Neural Network Inference

CORDIC-Based Tanh Activation on Artix-7 FPGA

Unnath (IMT2023620) and Hrushikesh (IMT2023619)

IIIT Bangalore

November 25, 2025

Project Goals & Platform

Objective

Deploy a complete MLP (784→64→10) with **real Tanh activation** on hardware.

- **Platform:** Xilinx Artix-7 (XC7A35T)
- **Board:** Digilent Basys 3
- **Input:** 4-bit Switch Index
- **Output:** LED Prediction

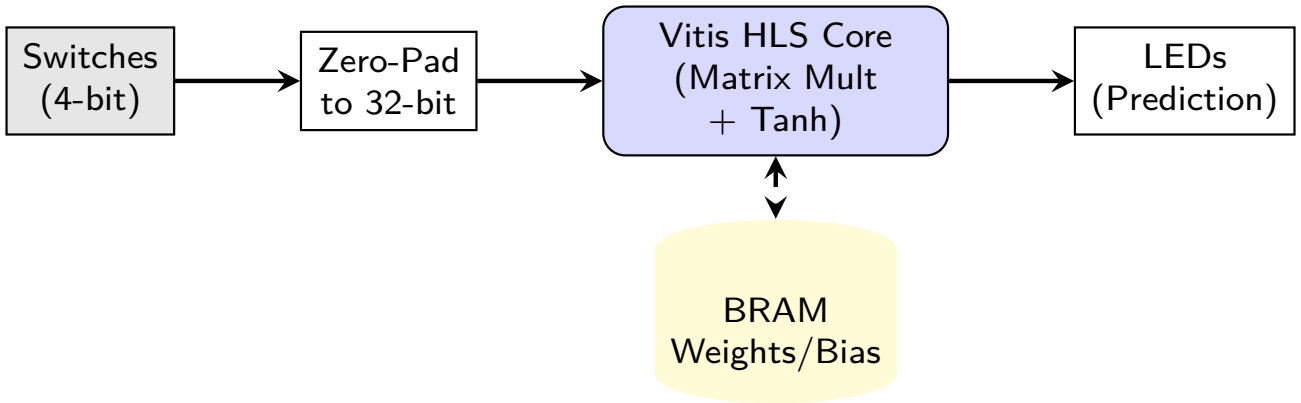
Key Challenge

Implementing the non-linear activation:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

using Fixed-Point math and CORDIC algorithms on an FPGA without FPU.

Data Flow Architecture



- **Input:** User selects image index via switches.
- **Processing:** Fully pipelined Matrix-Vector Multiplication.
- **Activation:** CORDIC-based iterative Tanh approximation.

Vivado Block Design

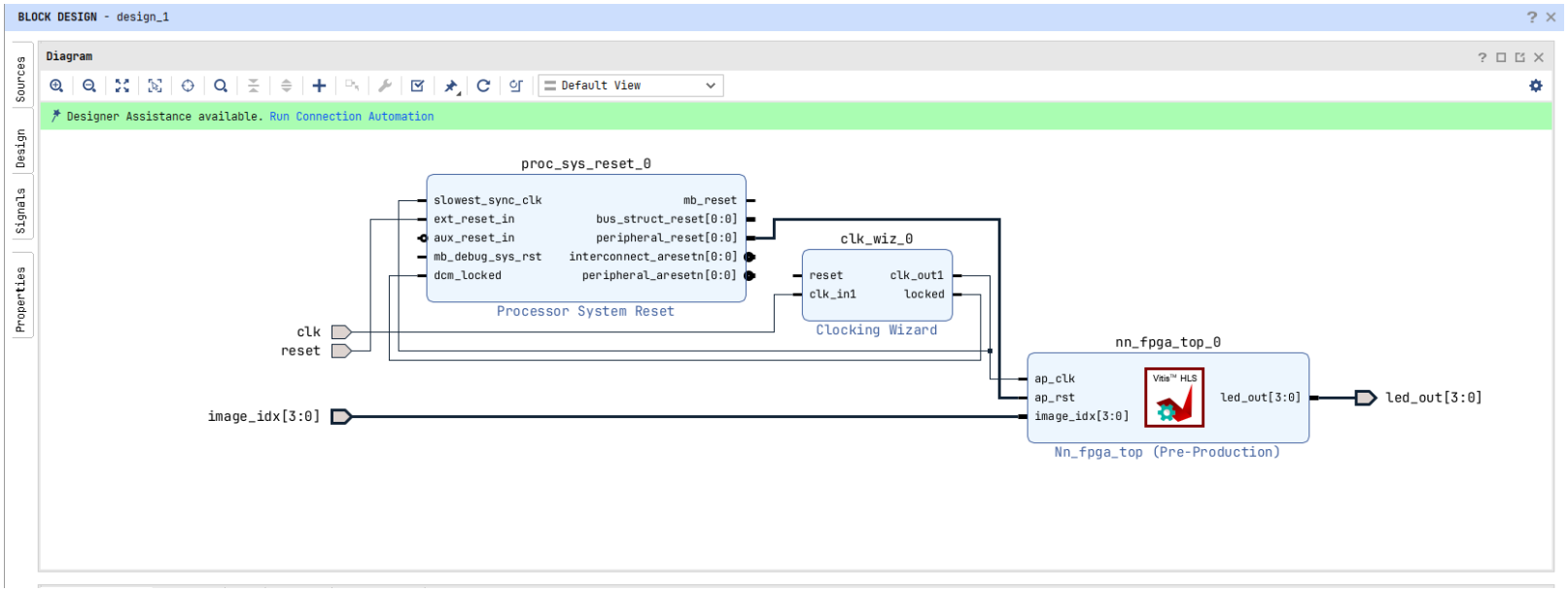


Figure 1: Vivado IP Integrator Block Design

- **Top Level:** Integration of the generated Vitis HLS Neural Network IP.
- **RTL Wrapper:** Maps physical board constraints (Switches/LEDs) to the Core interfaces.

Design Flow: From HLS to Bitstream

1. **Vitis HLS Development:**
   - Defined Top Function: `nn_fpga_top`.
   - Wrote C++ Testbench to validate logic against reference data.
   - Exported weights from Python; configured `.h` headers to force **BRAM** inference (avoiding register overflow).
2. **Verification & Synthesis:**
   - **C Simulation:** Verified bit-exact functional correctness.
   - **C Synthesis:** Generated RTL (Verilog) & analyzed resource usage.
   - **C/RTL Co-Simulation:** Verified RTL cycle-accurate behavior.
3. **IP Export:**
   - Packaged the synthesized design as a ZIP file (`Export RTL`).
4. **Vivado Integration:**
   - Imported IP ZIP into Vivado IP Catalog.
   - Instantiated in Block Design and generated Bitstream.

Fixed-Point & CORDIC Strategy

```
1  // Fixed-Point Type (Q8.8)
2  typedef ap_fixed<16, 8> data_t;
3
4  // Tanh Activation
5  data_t my_tanh(data_t x) {
6      #pragma HLS INLINE
7      // Vitis HLS infers CORDIC
8      // for hyperbolic functions
9      return hls::tanh(x);
10 }
```

Quantization (Q8.8)

- **Range:** $[-128, 127.9]$
- **Precision:** $2^{-8} \approx 0.0039$

Cost of Accuracy

The HLS Tanh implementation consumes **37 DSP48E1** slices solely for the iterative CORDIC math!

Timing Closure Analysis

Table 1: Timing Metrics

Metric	Value
Target	10.00 ns
WNS	**+1.916 ns**
WHS	+0.018 ns

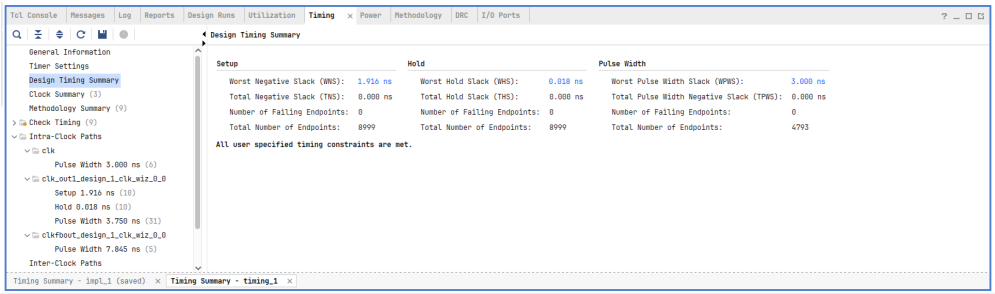


Figure 2: Vivado Timing Summary

Max Frequency

$$T_{min} = 8.084 \text{ ns}$$

$$F_{max} \approx \mathbf{123.7} \text{ MHz}$$

Critical Path Schematic

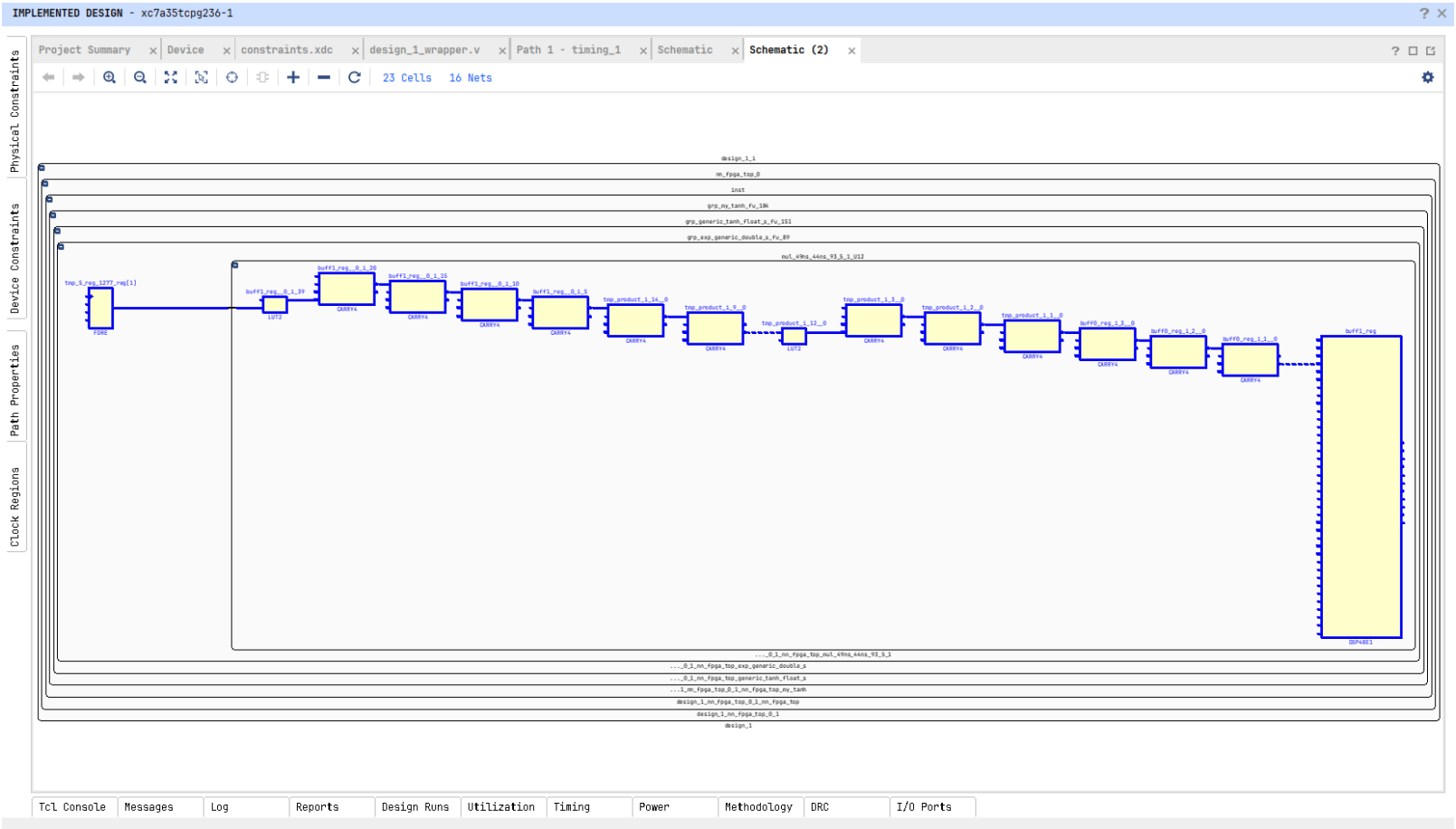


Figure 3: The critical path passes through a chain of DSP slices performing the iterative exponential calculations required for the hyperbolic tangent.

Inference Latency: FPGA vs CPU

HLS Synthesis Report:

- Latency: **5.840E5 ns**
- Calculation:

$$5.84 \times 10^5 \text{ ns} = \mathbf{0.584} \text{ ms}$$

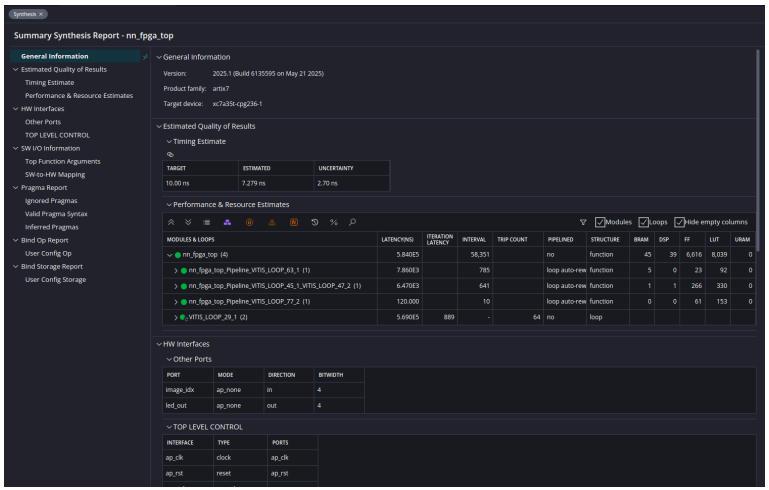


Figure 4: HLS Latency Report

Table 2: Comparison

Metric	**FPGA**	**CPU**
Latency	0.584 ms	0.026 ms
Power	**0.298 W**	∼65 W
Clock	100 MHz	∼4 GHz

CPU Logs:

- Mean Latency: **0.026 ms**

Resource & Power Utilization

Resource Usage (Post-Route):

Table 3: Utilization

Resource	Used	Util %
LUT	4,262	20.49%
FF	4,308	10.36%
BRAM	23	46.00%
DSP	**40**	**44.44%**

Power Consumption:

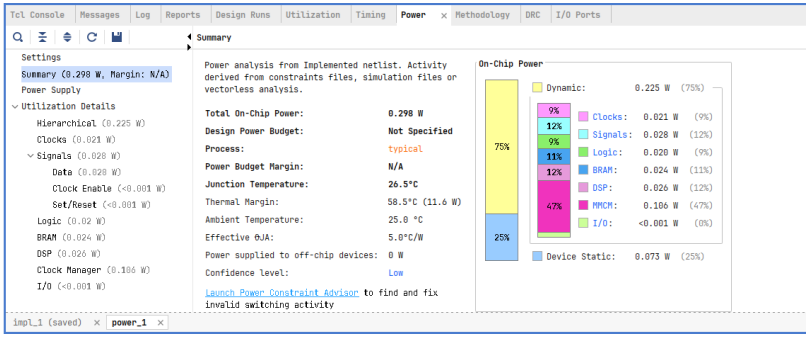


Figure 5: Vivado Power Report

- **Total On-Chip:** 0.298 W
- **Dynamic:** 0.225 W

Verification Results (Full Test Batch)

Table 4: FPGA vs C Sim vs PC Inference

ID	FPGA	C Sim	PC	Label	Match
0	7	7	7	7	✓
1	2	2	2	2	✓
2	1	1	1	1	✓
3	0	0	0	0	✓
4	4	4	4	4	✓
5	1	1	1	1	✓
6	4	4	4	4	✓
7	9	9	9	9	✓
8	6	6	6	5	X
9	9	9	9	9	✓

Accuracy Statistics

- **Full Test Set:** 1000 images
- **Final Accuracy:** 95.80%

Consistency Check

Hardware (FPGA), C Simulation, and Python Fixed-Point Model (PC) produce identical results, confirming the reliability of the HLS logic.

Project Resources

GitHub Repository

Contains Vitis HLS Source Code, Testbench, and Exported IP Core (.zip).

`https://github.com/AspiringPianist/final_nn_fpga_tanh`

Hardware Implementation Files

Contains Vivado Block Design, Bitstream, and Constraints.

- Archive: `vivado_final_nn_fpga_tanh.zip`

Thank You!

Questions?